

<김기환 / >

Software Engineer

이력서와 포트폴리오를 하나로 묶어, HCI와 Visual Analytics 연구에서 시작해 AI 시스템, 제품 플랫폼, 내부 운영 자동화로 이어진 작업을 정리했습니다. 복잡한 시스템을 사람이 이해하고 사용할 수 있게 만든 프론트엔드 구현과 브라우저 자동화 작업을 중심으로 담았습니다.

01 Curriculum Vitae · 이력서

경력, 프로젝트, 학력, 연구 및 논문 이력

02 Portfolio · 포트폴리오

내부 운영 자동화, 서버 주도 UI, 렌더링 최적화, AutoML·예측 설명 UI, 추천 시스템 실험

<김기환 / >

Summary

소프트웨어 엔지니어로 7년째 일하고 있습니다.

실무를 시작하기 전에는 대학원에서 HCI와 Visual Analytics를 연구하며, 복잡한 정보와 시스템을 사용자가 이해하기 쉬운 화면으로 옮기는 문제를 다뤘습니다. 실무에서는 AutoML과 설명 가능한 AI(XAI), 데이터 시각화, 콘텐츠 플랫폼, 서버 주도 UI, 내부 운영 자동화에서도 같은 문제를 이어왔습니다.

최근에는 AI를 활용해 사람이 더 적은 노력으로 더 나은 판단을 할 수 있게 만드는 문제에 관심이 있습니다. GitHub 중심 협업, 브라우저 자동화, 실행 가능한 프로토타입을 통해 여러 도구에 흩어진 정보와 근거를 더 쉽게 다루는 방식을 실험하고 있습니다.

<https://kihwan.kim> juljin1875@gmail.com [LinkedIn](#) [GitHub](#)

Themes

Human-System Interaction

석사 때 HCI와 Visual Analytics를 연구했습니다. 모델이 내놓은 답을 사람이 화면에서 따져볼 수 있게 만드는 일입니다.

Understandable AI Systems

AutoML 제품의 예측 설명 UI를 만들었습니다. 숫자만 던져주면 비전문가는 그 값을 쓸지 말지 판단할 수 없어서, 왜 그렇게 나왔는지를 함께 보여줬습니다.

Product Experimentation Infrastructure

화면을 고치거나 A/B 테스트를 돌리는 데 배포를 기다리지 않게 했습니다. 서버 주도 UI로 옮겼고, 기존 화면은 한 번에 갈아엎지 않고 점진적으로 넘겼습니다.

AI-Native Development Workflow

아이디어는 글로 설명하기보다 눌러볼 수 있는 화면으로 보여주는 게 빠릅니다. 에이전트와 같이 만들면 그 사이가 하루 이틀로 줄어듭니다.

Collaboration and Operations Tooling

이슈, PR, 로그, 대화가 저마다 다른 화면에 갇혀 있어서 그 사이를 잇는 건 사람 몫이 됩니다. 그 몫을 GitHub 워크플로와 사내 도구 연동, 브라우저 자동화로 옮겼습니다.

Experience

Levels

프론트엔드 엔지니어

2025.07 – 현재

AI-Native 제품 개발과 내부 운영 자동화를 실험하고 있습니다. GitHub 중심 협업과 동작하는 프로토타입으로 제품 변경안을 빠르게 확인하고, 브라우저 자동화로 반복적인 CS/QA 확인을 줄입니다.

React

Next.js

TypeScript

TanStack Query

Zustand

SSE

Playwright

Sentry

Mixpanel

ChannelTalk

PageAgent

Generative UI

AI 캐릭터 챗 서비스

2026.05 – 현재

AI 캐릭터와 대화하며 이야기를 이어가는 서비스와 캐릭터 콘텐츠를 작성·심사하는 Admin을 만들고 있고, 출시를 준비 중입니다.

- 진행 중인 스트리밍 응답과 저장된 대화를 분리하고 전송·완료·실패·재생성 상태 구현
- 스토리·로어북·검색 키워드 등 캐릭터 콘텐츠의 작성·심사 화면 구현
- 기획 문서, Figma, Swagger에서 다르게 정의된 부분을 정리하고 구현 기준과 QA 시나리오 작성

Vuddy 커머스 [↗](#)

2025.07 – 현재

Vuddy의 소비자 웹, Creator Studio, 내부 Admin을 구현하고 운영했습니다. 클래스 탐색과 결제부터 주문, 취소·반품·교환까지 커머스 흐름을 맡았습니다.

- 클래스 탐색·영상·결제, Studio 주문 관리, Admin 상품·수강생·이용권 관리 구현
- 장바구니, 본인인증, 2차 창작자 신청과 취소·반품·교환 정책 변경 반영
- HLS 홈 캐러셀과 모바일 WebView의 safe area·스크롤·뒤로가기 문제 개선

PageAgent 기반 내부 운영 자동화

2026.03 – 현재

Case study [↗](#)

CS/QA 담당자가 여러 내부 도구를 오가며
확인하던 일을 브라우저 자동화로 줄였습니다.
자연어 요청을 실행 절차로 바꾸고, 결과와 근거를
한 화면에 정리했습니다.

- 계정, 결제, 매핑 정보를 여러 내부 도구에서 모아 한 화면에서 비교할 수 있게 구성
- Chrome Managed Profile 환경에 맞게 Alibaba PageAgent를 확장하고 탭 이동, 클릭, 입력 등 화면 조작을 자동화
- LLM 응답과 실제 실행 절차를 분리해 같은 요청을 다시 확인할 수 있게 구성
- CS/QA의 반복적인 테이블 조회와 교차 확인을 자동화

리디주식회사

프론트엔드 엔지니어

2022.05 – 2025.07

서버 주도 UI와 점진적 마이그레이션으로 운영 UI 변경과 A/B 테스트를 배포 주기와 분리하고 실행 속도를 높였습니다.

Next.js

React

TypeScript

Emotion

Jest

PHP

Twig

Sentry

Virtualization

리디 웹 플랫폼 [↗](#)

2022.05 – 2025.07

Case study [↗](#)

서버 주도 UI와 아일랜드 아키텍처로 레거시 페이지를 React로 점진적으로 이전했습니다. 운영 UI 변경과 A/B 테스트를 더 빠르게 실행했습니다.

- 서버 주도 UI를 클라이언트에서 해석해 화면 변경과 배포 주기 분리
- PHP 페이지를 React 구조로 점진적으로 이전해 마이그레이션 위험 완화
- A/B 테스트와 운영 UI 변경을 더 유연하게 실행
- 모바일, 데스크탑, 특수 디바이스 환경 안정성 유지

대규모 리스트 렌더링 최적화

2022.05 – 2025.07

Case study [↗](#)

대량 콘텐츠 탐색에서 렌더링 병목이 사용자 경험을 해치지 않도록 리스트 구조와 로딩 경험을 개선했습니다.

- 대량 DOM 렌더링의 프레임 드랍 분석
- windowing으로 화면 영역만 렌더링
- 스크롤 위치와 동적 높이 처리
- skeleton UI로 초기 로딩 경험 개선

사용자 환경 안정화

2022.05 – 2025.07

브라우저, 디바이스, 확장 프로그램 차이로 생기는 사용자 환경의 불안정성을 줄이고 운영 대응력을 높였습니다.

- 실제 사용자 오류를 Sentry로 추적
- 번역 플러그인 등 외부 extension 충돌 대응
- E-ink 렌더링과 UX 이슈 개선

티맥스엔터프라이즈

연구원

2020.02 – 2022.05

現 티맥스비즈에이아이

舊 티맥스데이터

비전문가도 AI/분석 기능을 사용할 수 있도록 AutoML 플랫폼의 사용 과정, 시각화, 인터랙션을 설계했습니다. 예측값과 근거를 함께 확인할 수 있는 UI를 구현했습니다.

React

TypeScript

Material-UI

Sass

Python

jQuery

레거시 시스템 React 전환

2021.01 – 2022.05

용도와 책임이 뒤섞인 사내 UI 라이브러리를 정리하고, 레거시 위젯을 React 컴포넌트로 대체했습니다.

- 기존 UI 코드를 용도별로 분리
- 레거시 위젯을 대체할 컴포넌트 신규 개발

AutoML 플랫폼

2020.02 – 2022.05

Case study [↗](#)

모델 개발 경험이 없는 사용자를 위한 Codeless 개발 스튜디오와 예측 근거 시각화를 구현했습니다.

- 비전문가용 Codeless 개발 스튜디오 구현
- 예측 근거 시각화와 대시보드 개발
- AutoML 엔진과 모델 개발자 간 인터페이스 연구
- 초기 제품의 요구사항과 구조 구체화

Open Source

OpenMMO

Contributor

2026.07 - 현재

AI 에이전트와 사람이 같은 프로토콜로 플레이하는 오픈소스 MMORPG(Rust · Svelte · Three.js · WebAssembly)에 기여하고 있습니다. 게임 개발은 처음이었고, 클라이언트 테스트로 시작해 지금은 Rust 서버 쪽을 주로 봅니다. 올린 PR 10건은 모두 병합됐습니다.

TypeScript

Svelte

Rust

WebAssembly

Vitest

클라이언트

- 반대하던 메인테이너를 설게 제안서로 설득해 HUD 미니맵 구현, 서버는 고치지 않고 이미 있던 지형 타일과 회전 규약을 그대로 사용. 위치를 계속 물어봐야 하는 파티원 마커는 리뷰를 받아 v1에서 제외하고, 그 과정에서 찾은 상태 갱신 누락은 원인 쪽에서 수정 ([#11](#), [#77](#))
- 낚시 상태 전이와 수면 높이 샘플링, 클릭 인텐트 분기의 테스트 공백 보완. 사거리 밖의 물 클릭은 이동으로 바꾸고 사거리 값을 WebAssembly로 노출해 서버와 클라이언트가 같은 값을 참조, 세션이 끊길 때 뒤늦게 올리던 예약 사운드도 취소 ([#56](#), [#64](#), [#65](#))

서버

- 지형 타일 캐시가 쌓이기만 하고 줄지 않아 세대 단위로 쓸어내고 용량 상한 적용, 읽기 경로는 락 없이 그대로 유지 ([#68](#))
- 운영자용 계정 밴/해제 구현, 캐릭터가 아닌 계정으로 끊어 부캐 재접속을 막고 검사를 지난 접속이 그대로 들어오는 틈은 기존 세션 락으로 차단 ([#95](#))
- 상인 NPC의 취침 스케줄은 에이전트 쪽 데이터라 서버가 볼 수 없어, 서버에도 보이는 침대 점유 상호작용으로 수면을 판정해 잠든 상인과의 거래를 거부. 모든 거래 경로가 이미 지나는 검증 한 곳에 넣어 자기 전에 열어둔 상점 창으로 결제되는 경우까지 차단 ([#103](#))
- 밤에는 상인이 자서 마을에 거래처가 없어, 어두워지면 좌판을 여는 야간 상인을 추가. 상인 정의가 두 파일에 나뉘어 있어 한쪽만 넣으면 띄울 수 없는 상점이 생기므로 그 불일치를 잡는 테스트를 함께 작성 ([#111](#))

문서·온보딩

- 지형을 굽기 전에는 화면이 까맣게 나오는 온보딩 공백을 직접 겪고 개발 셋업 문서에 반영 ([#78](#))
- CI가 검사하는 명령들이 워크플로 파일에만 있고 착수 전 확인할 것도 흩어져 있어, 새 기여자가 CI 실패나 중복 작업으로 배우던 것들을 CONTRIBUTING 가이드로 모음. 세 언어 README에서 링크 ([#118](#))

vite-plus

Contributor
2026.08

Vite·Vitest를 만든 VoidZero의 툴체인 vite-plus(Rust · TypeScript 하이브리드)에 기여. 셸 통합이 작업 디렉토리 플래그를 놓치던 문제를 찾아 Rust 쪽에서 고치고 스냅샷 테스트로 덮어 병합했습니다.

Rust

Shell

셸 통합

- 생성되는 셸 래퍼가 전역 `-C <dir>` 플래그(`-C <dir> · -C<dir> · -C=<dir>`) 뒤에 오는 `vp env use`를 인식하지 못해 환경 변경이 현재 셸 세션에 적용되지 않던 문제를 `setup.rs`에서 수정. `vpr` 자동완성도 `zsh · Fish · Nushell · PowerShell`에서 작업 디렉토리 옵션을 보존하도록 맞추고, 네 셸의 통합 템플릿을 스냅샷 테스트로 고정 ([#2508](#), 이슈 [#2056](#))

Education

울산과학기술원 (UNIST)

컴퓨터공학 석사

2018.03 – 2020.02

기술경영학 학사 (컴퓨터공학 융합전공)

2013.03 – 2018.02

AI 기반 UI와 추천 시스템 연구 [↗](#)

2019.01 – 2020.02

Case study [↗](#)

추천 시스템의 탐색용 아이템 노출을 사용자가 어떻게 이해하는지 연구했습니다. 더 나은 피드백을 끌어내는 설명 방식을 실험했습니다.

- Explore-Exploit 문제에서 투명성 효과 측정
- 탐색 아이템 노출 방식과 사용자 인식 분석
- 사용자 로그 가치 평가 지표 제안
- 웹 기반 영화 추천 실험 시스템 구현
- 사용자와 추천 알고리즘의 상호작용을 관찰하는 UI 실험 설계

Python

Flask

Surprise

jQuery

웹 사용자 행동 모델링 [↗](#)

2019.02 – 2020.02

웹 환경의 사용자 의사결정과 행동 패턴을 모델링했습니다. 사람이 시스템을 이용하는 방식을 데이터로 분석했습니다.

- 역강화학습으로 사용자 보상 함수 추정
- 데이터 분석, 역사 교육, 카드 게임 환경을 다룸
- Tableau와 위키피디아 기반 교육 도구 활용

Python

TensorFlow

Keras

scikit-learn

Publications

An Empirical Analysis on Transparent Algorithmic Exploration in Recommender Systems [↗](#)

김기환

A Computing Research Repository (CoRR), 2108.00151, 2021

- 추천 시스템이 사용자 취향을 파악하기 위해 랜덤 아이템을 노출하는 방식 연구
- 실험용 웹 기반 영화 추천 시스템 구현
- 사용자 로그의 학습 가치 평가 지표 제안
- 추천 시스템의 Explore-Exploit 문제에서 투명성이 데이터 품질에 미치는 영향 측정
- Amazon MTurk에서 94명의 피험자 모집
- 넷플릭스와 유사한 실험 환경에서 실사용 로그와 설문 응답 수집

ST-GRAT: A Novel Spatio-temporal Graph Attention Networks for Accurately Forecasting Dynamically Changing Road Speed [↗](#)

박천복, 이충기, 방효진, 태윤원, 김기환, 진승민, 고성안, 주재걸

ACM International Conference on Information and Knowledge Management (CIKM), 2020

- 한국도로공사의 도로 바닥에 설치된 차량 감지 센서 데이터를 전처리
- Attention이 효과적으로 동작한 경우를 패턴별로 분류

Modeling Exploration/Exploitation Decisions through Mobile Sensing for Understanding Mechanisms of Addiction [↗](#)

김기환, 김상훈, 이충기, 고성안

ACM International Conference on Mobile Systems, Applications, and Services (MobiSys), 2019

- 스마트폰 사용 로그와 Inverse Reinforcement Learning으로 중독 질환 여부 감지 시스템 제안

An Empirical Study on the Relationship Between the Number of Coordinated Views and Visual Analysis [↗](#)

오주영, 이충기, 김휘연, 김기환, 권오상, Eric D. Ragan, 권범철, 고성안

A Computing Research Repository (CoRR), 2204.09524, 2018

- 시각화 차트 수가 데이터 분석 과정에 미치는 영향 실험
- 44명의 피험자에게 시각적 분석 도구와 데이터 분석 과제 제공
- Think-aloud 프로토콜, 화면 녹화, 로그 데이터로 사용자 분석 패턴 분류
- 차트 수와 과제 점수 간 양의 상관관계 관찰

시각화 기반 딥러닝 분석 기술 [↗](#)

이재성, 김기환, 이충기, 고성안

소음진동 제27권 제6호 2017.11

- 딥러닝 모델을 해석하기 위한 시각화 기법 정리 및 조사

PageAgent 기반 내부 운영 자동화

여러 내부 도구와 외부 서비스를 오가던 CS/QA 확인 업무를 PageAgent 기반 브라우저 자동화로 처리하고, 결과와 근거를 한 화면에서 확인할 수 있게 한 작업입니다.

PageAgent

Result UI

Browser Automation

QA/CS Automation

Internal Operations

Problem

카카오 계정으로 구매한 사용자가 네이버 계정으로 로그인해 구매 내역이 보이지 않는 CS가 들어오면, 운영자는 여러 도구를 오가며 구매 귀속 계정을 추적해야 했습니다.

- 소셜 로그인 계정 간 구매 귀속 확인
- 유저, 결제, 계정 매핑 테이블의 순차 조회
- 배송 이슈에서는 택배사 외부 서비스까지 조회
- 운영자가 매번 조회 순서를 직접 판단해야 하는 부담

Initial Hypothesis

요청마다 확인할 정보와 순서가 달라지기 때문에, 자연어 요청에 맞춰 조회 절차와 결과 화면을 구성하면 확인 시간을 줄일 수 있다고 보았습니다.

- 정적인 어드민 화면을 추가하는 대신 상황별 조회 절차 구성
- 계정 후보, 결제 내역, 매핑 결과를 한 화면에서 정리
- 운영자가 다음에 볼 테이블과 순서를 직접 판단하는 부담 감소

Why UI Generation Was Not Enough

실제로는 UI를 새로 만드는 것보다 기존 사내 도구를 오가며 정확하게 조회하고 조작하는 능력이 더 중요했습니다.

정보가 여러 어드민과 탭에 분산

조회와 입력은 여전히 사람이 수행

여러 서비스 이동과 조작이 중요

Key Insight

내부 도구에서 중요한 것은 새 UI를 만드는 일이 아니라, 여러 시스템의 조회와 조작을 안정적으로 연결하는 일이다.

Approach

PageAgent를 기반으로 자연어 요청을 실행 절차로 바꾸고, 브라우저에서 기존 UI를 조작한 뒤 근거와 결과를 운영자가 확인하기 쉬운 형태로 정리했습니다. 익스텐션 기반으로 탭마다 분리된 브라우저 컨텍스트의 데이터를 자동화 코드에서 이어서 처리했습니다.

Natural Language

운영자 요청

LLM

실행 절차 생성

PageAgent

UI 조작 실행

Browser

여러 탭 이동

Result UI

정보 재구성

Example Scenario

위와 같은 CS 확인 업무에서, 자동화 전후의 작업은 다음처럼 달라집니다.

Before

- 1. 문의 정보에서 후보 사용자 식별
- 2. 유저 어드민에서 소셜 계정 확인
- 3. 계정 매핑 테이블에서 동일 사용자 후보 탐색
- 4. 결제 어드민에서 구매 귀속 계정 조회
- 5. 필요 시 택배사 등 외부 서비스까지 조회
- 6. 여러 탭의 결과를 사람이 직접 종합

After

- 1. 운영자가 자연어로 확인 요청
- 2. 자동화 코드가 필요한 조회 순서 생성
- 3. PageAgent가 여러 탭에서 클릭과 입력 수행
- 4. 구매 귀속 계정과 현재 로그인 계정의 차이 요약
- 5. CS 답변에 필요한 결과 UI 제공

Architecture

- **LLM:** 사용자 명령을 해석하고 실행 절차 생성
- **PageAgent:** 브라우저 탭 이동, 버튼 클릭, 입력 등 실제 UI를 조작하고, 탭별 데이터를 자동화 코드에서 연결
- **Chrome Managed Profile:** 사내 도구 접근 권한과 인증 세션을 유지하고, 브라우저 익스텐션 형태로 Agent를 배포하는 채널
- **UI Layer:** 수집한 정보를 운영자가 판단하기 쉬운 결과로 재구성

Implementation Scope

PageAgent의 브라우저 조작 엔진을 새로 만든 것이 아니라, 내부 운영 도구에 맞게 연결하고 업무 절차를 검증했습니다.

- **Task Mapping:** CS/QA 요청을 PageAgent가 수행 가능한 실행 절차로 변환하는 업무별 규칙 정의
- **Selector Strategy:** 반복 사용 UI는 전용 selector로 연결해 시각 탐색 의존도를 낮추고 조작 안정성 강화
- **Extension Context:** 탭마다 분리된 브라우저 컨텍스트를 운영 업무 단위로 이어 붙이는 구조 검토
- **Result Contract:** 수집 근거와 요약 결과를 분리해 운영자가 확인하기 쉬운 결과 UI로 구성

Impact

반복 확인 단순화

내부 어드민과 외부 서비스를 오가던 QA/CS
확인 업무를 자동화할 수 있는 절차로 정리

기존 UI 유지

기존 어드민 위에서 브라우저 자동화를
적용하는 방식 검증

재현성 강화

전용 selector로 화면 요소 탐색 의존도
감소

What I Learned

- 새 UI를 생성하는 접근만으로는 내부 운영 도구의 문제를 해결하기 어렵다.
- 실제 UI 조작과 결합했을 때 업무 자동화 가치가 커진다.
- 내부 도구에서는 정확성과 재현성, 나중에 확인할 수 있는 실행 절차가 중요하다.
- UI를 새로 만드는 것보다 기존 시스템을 안정적으로 연결하고 조작하는 능력이 더 중요하다.

RiGrid 기반 서버 주도 UI

콘텐츠 플랫폼의 UI 구성과 비즈니스 데이터를 분리하고, Grid/Cell 단위로 재사용 가능한 클라이언트 렌더링 구조를 설계한 작업입니다.

서버 주도 UI

RiGrid

GraphQL

Multi-platform UI

Problem

기존 콘텐츠 UI는 비즈니스 데이터와 표시 로직이 강하게 결합되어, 레이아웃이나 섹션 구성이 바뀔 때마다 여러 클라이언트 코드 수정이 필요했습니다.

- 플랫폼별로 유사한 UI 로직이 중복 구현됨
- 운영 실험이나 콘텐츠 구성을 바꾸기 위해 앱/웹 배포가 필요함
- 작품 상세, 랜딩, 검색처럼 서로 다른 화면에서 재사용 가능한 구조가 부족함

Approach

서버가 Grid와 Cell 구조를 내려주고, 클라이언트는 schema에 맞춰 Cell tree를 재구성한 뒤 대응되는 컴포넌트를 렌더링하는 구조로 접근했습니다.

- GraphQL schema 기반 Grid/Cell 인터페이스 정의
- Cell의 layout과 type으로 UI 표현과 데이터 바인딩을 분리
- AsyncCell, ChildCells, FlexColumnCell 등 유틸리티 Cell로 조합성 보강

Implementation Focus

Client Renderer

서버 응답의 Cell 목록을 트리로 재구성하고, layout/type에 맞는 컴포넌트로 매핑

Compatibility

기존 클라이언트가 깨지지 않도록 추가 중심의 schema 변경과 점진적 전환 고려

Operational Flexibility

Cell 단위 조합으로 랜딩/검색/상세 화면의 콘텐츠 구성과 실험 가능성 확대

Impact

- UI 데이터와 비즈니스 데이터를 분리해 변경 영향 범위를 줄이는 방향 검증
- 기획, 디자인, 개발이 Cell 단위 구조를 공유하며 협업할 수 있는 공통 언어 확보
- 마케팅 랜딩 페이지와 검색 페이지 등 다양한 화면으로 서버 주도 UI 적용 범위 확장
- 부분 전환 과정에서 middleware 병목과 성능 저하를 확인하고, 전체 전환 필요성을 정리

대규모 콘텐츠 렌더링 최적화

예상보다 큰 콘텐츠 목록을 안정적으로 렌더링하기 위해 DOM 수, 네트워크 페이로드, 사용자 상호작용 지연을 함께 줄인 프론트엔드 성능 개선 작업입니다.

- Virtualization
- React
- Performance
- Large Lists

Problem

250개 작품 기준으로 만들어진 페이지가 실제 이벤트에서 6,000개 이상의 작품을 다루게 되면서 렌더링과 상호작용 성능이 급격히 나빠졌습니다.

- 대량 DOM 렌더링으로 저사양 기기에서 초기 표시가 지연됨
- 스켈레톤과 실제 아이템 렌더링 책임이 한 컴포넌트에 섞임
- 필터 변경 시 네트워크 재요청과 메인 스레드 부하가 함께 발생함

Approach

화면에 보이는 아이템만 렌더링하도록 리스트 가상화를 적용하고, 스켈레톤 표현과 데이터 필터링 흐름을 분리해 체감 성능을 개선했습니다.

- window scroll과 결합한 list virtualization 적용
- SVG background 기반 skeleton으로 렌더링 책임 분리
- gzip 압축과 클라이언트 필터링 구조 검토

Implementation Focus

Rendering Budget	Skeleton Strategy	Interaction Cost
전체 DOM을 한 번에 그리지 않고 viewport 주변 아이템만 유지하도록 구조 변경	스켈레톤을 반복 background로 처리해 가상화 컴포넌트의 관심사를 단순화	필터링 중 체감 로딩과 실제 데이터 처리 비용을 함께 분석하고 개선

Impact

- 대량 콘텐츠 목록에서도 초기 렌더링과 스크롤 체감 성능을 유지할 수 있는 구조 확보
- 스켈레톤 표시와 실제 아이템 렌더링을 분리해 컴포넌트 책임을 명확하게 정리
- 큰 JSON payload에 gzip을 적용해 전송 크기를 약 1/8 수준까지 줄일 수 있음을 확인
- 필터링 로직에서 네트워크 비용과 CPU 비용을 함께 봐야 한다는 기준을 정립

Enterprise AutoML 예측 설명 UI 개발

정형 데이터 기반 AutoML 모델을 기업 의사결정에 활용할 수 있도록, 모델 단위와 개별 예측 단위의 설명 UI를 구현한 작업입니다.

AutoML

예측 설명 UI

Tabular Data

Decision Tree

What-if UX

Problem

ERP, HR, CRM처럼 정형 데이터가 이미 축적된 영역에서는 AutoML 적용 가능성이 높지만, 기업 사용자가 예측값만 보고 의사결정에 반영하기는 어려웠습니다.

- 정확도가 높아도 왜 그런 예측이 나왔는지 설명할 수 있어야 함
- 인사평가, 채용, 수요 예측처럼 설명 책임이 필요한 업무가 존재
- ML 비전문가가 모델을 검증하고 배포 여부를 판단할 수 있는 UX가 필요

Approach

전체 모델의 주요 변수는 Feature Importance로 설명하고, 개별 예측은 Local Surrogate Model과 대표 인스턴스를 통해 설명하는 구조로 접근했습니다.

- Decision Tree/Ensemble 모델의 해석 가능성과 사용성 한계 분석
- 클러스터링 기반 prototype sampling으로 대표 데이터 인스턴스 추출
- 사용자가 입력값을 바꾸며 예측 변화와 근거를 확인하는 What-if 흐름 구성

Implementation Focus

Global Explanation

Feature Importance 기반으로 모델 전체에서 중요한 변수를 정렬하고 시각화

Local Explanation

개별 예측을 설명하기 위해 surrogate model과 feature contribution을 제공

Validation UX

대표 인스턴스와 직접 입력한 행 데이터로 예외 케이스를 검증하는 인터페이스 구성

Impact

- AutoML을 예측 자동화에 그치지 않고, 배포 전에 근거와 예외를 확인하는 제품 기능으로 확장
- ML 비전문가도 모델의 주요 판단 근거와 개별 예측의 이유를 확인할 수 있게 개선
- ERP/HR/CRM 등 기업 정형 데이터 기반 AI 기능의 적용 가능성과 검증 기준을 정리
- 예측값, 규칙 경로, 변수 기여도를 함께 보여주는 예측 설명 UI를 제품화

추천 시스템 탐색 UI 실험 플랫폼

추천 시스템의 Explore-Exploit 문제에서 탐색 아이템을 어떻게 설명해야 사용자 경험과 피드백 품질을 함께 개선할 수 있는지 검증한 연구/구현 작업입니다.

Recommender Systems

Experiment Platform

User Study

Flask

Python

Problem

추천 시스템은 사용자의 취향을 더 잘 파악하기 위해 때때로 취향과 맞지 않을 수 있는 탐색 아이템을 노출해야 하지만, 사용자는 이를 낮은 품질의 추천으로 인식할 수 있습니다.

- 탐색 아이템 노출이 사용자 신뢰와 피드백 품질에 미치는 영향이 불명확
- 추천 품질 개선을 위해서는 클릭/평점 로그의 학습 가치까지 함께 평가해야 함
- 실사용에 가까운 환경에서 UI 설명 방식과 사용자 행동을 함께 측정할 필요

Approach

넷플릭스와 유사한 웹 기반 영화 추천 실험 시스템을 구현하고, 탐색 아이템 노출의 투명성이 사용자 행동과 로그 가치에 미치는 영향을 측정했습니다.

- 영화 추천 목록, 상세 정보, 피드백 수집을 포함한 실험 UI 구현
- Amazon MTurk에서 94명의 피험자를 모집해 실사용 로그와 설문 응답 수집
- 추천 모델 학습 관점에서 사용자 로그의 가치를 평가하는 지표 제안

Implementation Focus

Experiment Platform

웹 기반 영화 추천 시스템을 구현해 추천 목록, 탐색 아이템, 피드백 수집 UI 제공

Transparent UX

사용자에게 낯선 추천이 왜 노출되는지 설명하는 UI 조건을 설계하고 비교

Data Quality Metric

수집된 로그가 추천 모델 개선에 얼마나 기여하는지 평가하는 지표 제안

Impact

- 추천 품질 문제를 알고리즘 성능뿐 아니라 사용자가 어떤 맥락에서 피드백을 남기는가의 문제로 확장
- AI 제품에서 모델 개선과 사용자 경험이 서로 연결된다는 점을 실험 시스템으로 검증
- 실사용 로그, 설문 응답, 추천 모델 평가 지표를 함께 다루는 실험 시스템 구축
- 대학원 연구 결과를 제품형 실험 플랫폼과 데이터 기반 UX 설계 사례로 정리